

먼저 핵심부터 잡으면, 이 코드의 NTTU는 $N = 2^{16} = 65536$ coefficient를 256×256 행렬처럼 보고, 256-point NTT를 다시 16×16 으로 분해해서 결국 16-point NTT를 네 번 수행하는 구조야. ntt_params.vh에도 target이 256×256 , "four passes of 16-point NTTs with transpose + twist in between"라고 직접 적혀 있어. 너희 RedactFormer 논문의 §4.3에 나오는 **16×16 tile, 4-stage radix-2 DIF, transpose, on-the-fly TF, Shoup + Barrett** 설명과도 실제 RTL이 정확히 대응된다.

1. 전체 계층부터 이렇게 외우면 돼

```

top_2dntt_256x256.v                                ← 최상위 NTT TOP
|
|—— phase1_256pe.v
|
|      |
|      |—— ntt16_buscore.v                          ← 16-point NTT #1
|      |      |—— q_const_rom_bank.v
|      |      |—— twiddle16_rom_multibank.v
|      |      |—— radix2_butterfly_dif.v × 32
|      |      |      |—— modular_adder              ⊃
|      |      |      |—— modular_subtractor         ⊃ modules.v
|      |      |      |—— barrett_mult.v
|      |
|      |—— transpose16x16_bus.v                      ← local 16×16 transpose
|      |
|      |—— single_on_fly_wrap.v                      ← Phase-1 twist
|      |      |—— q_const_rom_bank.v
|      |      |—— single_twist_rom.v
|      |      |      |—— twiddle16_rom_multibank.v
|      |      |      |—— single_on_fly.v
|      |      |—— shoup_mult.v ×16                  ← 다음 TF 생성

```

```

|      |      └── barrett_mult.v ×16    ← input × TF
|      |
|      └── ntt16_buscore.v                ← 16-point NTT #2
|
|── global_xpose_pp_256x256                ← top 파일 안에 있음
|      └── sram_bank_1p × (16 banks × 2 sets)
|
|── phase2_256pe.v
|
|      |── ntt16_buscore.v                ← 16-point NTT #3
|      |── transpose16x16_bus.v
|      |── double_on_fly_wrap.v
|      |      |── q_const_rom_bank.v
|      |      |── double_twist_rom.v
|      |      └── double_on_fly.v
|      |      |── shoup_mult.v ×16
|      |      └── barrett_mult.v ×16
|
|      └── ntt16_buscore.v                ← 16-point NTT #4

```

그래서 top-level dataflow만 보면 훨씬 간단해.

NTT16 → Local Transpose → Single OF – Twist → NTT16

이게 **Phase 1**.

그다음

Global 256 × 256 Transpose

한 번 하고,

$NTT16 \rightarrow Local\ Transpose \rightarrow Double\ OF - Twist \rightarrow NTT16$

이게 **Phase 2**야.

SHARP에서 설명하는 four-step NTT의 butterfly $\rightarrow$ transpose $\rightarrow$ twisting $\rightarrow$ butterfly와 OF-Twist가 이 구조를 이해하는 데 직접적인 배경이 된다.

참고로 파일명이 phase1_256pe라고 되어 있지만 **이 모듈 안에 실제 PE 256개가 instantiate되어 있는 건 아니야**. 이 이름 때문에 BTS의 256 PE 같은 식으로 생각하면 헷갈려. RTL상 애는 그냥 위 네 stage를 연결한 pipeline module이라고 생각하는 게 맞아.

2. 그런데 공부는 top부터 한 줄씩 읽으면 안 돼

처음 20분만 **top-down**으로 구조를 훑고, 실제 공부는 **bottom-up**으로 하는 걸 추천해.

가장 좋은 순서는 이거야.

단 계	먼저 볼 코드	돌릴 TB	여기서 이해할 것
0	ntt_params.vh $\rightarrow$ top $\rightarrow$ phase1/2	아직 X	전체 지도만 잡기
1	radix2_butterfly_dif.v	tb_radix2_bfly_hex.v	NTT butterfly 한 개
2	ntt16_buscore.v	tb_ntt16_buscore_levels.v	16-point NTT 전체
3	같은 ntt16_buscore.v	tb_ntt16_barrett.v	여러 modulus regression
4	transpose16x16_bus.v	tb_transpose16x16_bus.v	tile 데이터 재배열
5	single_on_fly.v $\rightarrow$ wrapper	tb_single_on_fly16_general.v	Phase1 twiddle 생성
6	phase1_256pe.v	tb_phase1_256pe_dc.v	네 stage를 연결해서 관찰
7	phase1_256pe.v	tb_phase1_256pe_gen.v	full 4096-frame

단 계	먼저 볼 코드	돌릴 TB	여기서 이해할 것
			pipeline
8	double_on_fly.v → phase2	전용 TB 없음	Phase2 / double OF-Twist
9	top_2dntt_256x256.v	전용 TB 없음	end-to-end

이 순서가 내가 보기엔 제일 좋다.

3. 첫날에는 ntt_params.vh부터

이 파일은 그냥 parameter 파일이 아니라 사실상 **미니 README**야.

여기서 당장 기억할 숫자는:

```
WORD_WIDTH = 60
LANES = 16
NUM_BANKS = 30
BARRETT_LAT = 4
```

그리고

$$\boxed{bank_id = \text{RNS modulus } q_i \text{ 선택}}$$

이라는 거야.

즉 bank_id=3이라고 PE 3번이 아니라,

$$[a(X)]_{q_3}$$

를 처리하는 데 필요한 q_3, μ_3 , twiddle table을 고르는 tag라고 보면 돼.

또 ntt_params.vh에서 아주 중요한 게 **bank가 바뀌면 1-cycle bubble을 넣으라는** 규칙이야. q와 twiddle ROM이 registered이기 때문이야.

4. 첫 번째 진짜 실습: radix-2 butterfly

먼저 radix2_butterfly_dif.v.

애 하나를 이해하면 된다:

$$\begin{aligned}A' &= A + B \pmod{q} \\ B' &= (A - B) \cdot w \pmod{q}\end{aligned}$$

코드를 보면 정말 그대로

A,B



야.

그리고 BARRETT_LAT=4라서 add 쪽도 4 cycle delay시켜 multiplication 결과와 timing을 맞춘다.

마지막 stage에서는 LAST_STAGE=1이고 $w = 1$ 이라서 Barrett multiplier 자체를 bypass한다. 이게 너희 논문에서 말하는 "stage 4 exploits unity twiddle $w = 1$ to bypass the multiplier" 그대로야.

여기서는 tb_radix2_bfly_hex.v를 돌려.

이 TB가 생각보다 잘 되어 있어. q width가 42/58/59/60인 대표 level들을 자동으로 찾고, 일반 butterfly와 LAST_STAGE bypass 둘 다 reference 계산과 비교해서 끝에

[PASS] All selected levels/sets/modes matched ...

를 내준다.

Wave에서는 처음엔

valid_in

A_in

B_in

w

q

add_mod

sub_mod

mul_res

A_out

B_out

valid_out

정도만 보면 충분해.

여기서 Barrett 내부 4 stage를 전부 이해하려고 하지 마. 처음 목표는 "butterfly가 이 수식을 4-cycle latency로 구현한다"까지만.

5. 두 번째: ntt16_buscore

여기가 NTT 공부의 핵심 모듈이야.

16-point NTT니까

$$\log_2 16 = 4$$

stage고, 각 stage에 8 butterfly씩:

$$4 \times 8 = 32$$

개의 radix2_butterfly_dif가 실제로 instantiate되어 있어.

대략:

16 inputs

Stage 0 : distance 8

(0,8), (1,9), ... (7,15)

Stage 1 : distance 4

Stage 2 : distance 2

Stage 3 : distance 1, w=1 bypass

16 outputs

야.

여기서는 **tb_ntt16_buscore_levels.v**를 먼저 추천해.

왜냐하면 범위를 굉장히 작게 줄일 수 있어.

ModelSim에서 예를 들면:

```
vsim work.tb_ntt16_buscore_levels +L0=0 +L1=0 +F0=0 +F1=0
```

이렇게 하면 **level 0의 frame 하나만** 볼 수 있어.

이게 PASS하면:

```
vsim work.tb_ntt16_buscore_levels +L0=0 +L1=0 +F0=0 +F1=15
```

그다음 여러 bank로 넓혀.

Wave에서는:

s_valid

bank_id

s_data

DUT.q_wire

DUT.v0

DUT.v1

DUT.v2

DUT.v3 또는 m_valid

m_data

를 보면 돼.

그러면 진짜로

$$input \xrightarrow{4cy} S0 \xrightarrow{4cy} S1 \xrightarrow{4cy} S2 \xrightarrow{4cy} S3$$

가 보인다.

TB에서도 NTT core latency를

$$4 \times BARRETT_LAT = \boxed{16 \text{ cycles}}$$

로 잡고 있어.

그리고 이 NTT는 DIF라 **output이 bit-reversed order**라는 것도 중요해.

6. tb_ntt16_barrett.v는 두 번째 regression용

이건 같은 ntt16_buscore를 검사하지만 조금 더 무겁고 디버깅 기능이 많아.

예를 들어:

```
vsim work.tb_ntt16_barrett +bank=0 +frames=1 +lanedbg=2
```

같이 하면 lane별 output도 자세히 볼 수 있어.

그러니까 역할을 나누면:

tb_ntt16_buscore_levels = 처음 구조 공부

tb_ntt16_barrett = 나중에 여러 bank/여러 frame 제대로 되는지 regression

이렇게 생각하면 된다.

7. 그다음 transpose

이제 transpose16x16_bus.v.

이건 산술이 아니라 데이터 이동을 공부하는 파일이야.

16 lane짜리 frame을 16 cycle 받으면

$$16 \times 16 = 256$$

개 데이터가 하나의 tile을 구성하지.

애는 두 개의 buffer를 ping-pong해서 한 tile을 받는 동안 이전 tile을 transpose해서 내 보내는 구조야.

tb_transpose16x16_bus.v가 이걸 꽤 잘 검증해. 외부 golden file이 없어도 자기 입력 pattern을 만들고 DUT가 어떤 mapping을 하는지 검사한다. 코드상 XOR-transpose / bit-reversed mapping / identity를 구별하는 검사까지 들어 있어.

따라서 이 TB는 **반드시 단독으로 한 번 돌려봐야 돼.**

Wave:

s_valid

s_data

DUT.w_row

DUT.wr_buf_sel

DUT.a_full

DUT.b_full

DUT.rd_active

DUT.rd_buf_sel

DUT.r_addr_q

m_valid

m_data

를 추천해.

이걸 보면 "16 cycle 쓰기 → buffer full → 다른 buffer 쓰면서 이전 buffer read"가 눈에

들어올 거야.

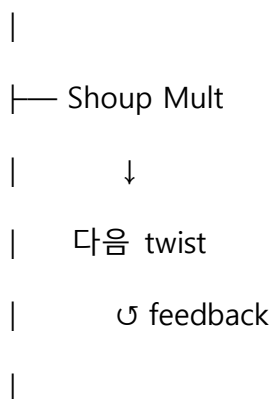
8. 그다음에 on-the-fly twist

여기가 이 코드에서 두 번째로 어려운 부분이야.

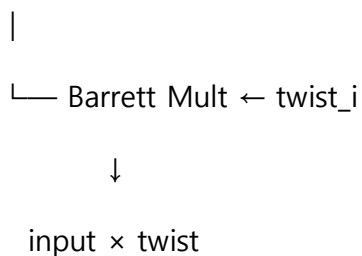
wrapper부터 읽지 말고 single_on_fly.v부터 읽어.

single_on_fly_twist16의 핵심은 lane마다:

twist_i



input_i



야.

즉 **Shoup** 쪽은 **TF sequence** 생성기, **Barrett** 쪽은 생성된 TF를 실제 데이터에 곱하는 **계산기**라고 분리해서 생각하면 된다.

초기에는:

seed_ctr=0 → even_seed

seed_ctr=1 → odd_seed

seed_ctr>=2 → Shoup 결과 feedback

식으로 시작하고, tile boundary마다 start가 다시 들어오면서 seed를 reload한다.

SHARP에서도 OF-Twist가 geometric sequence 성질을 이용해서 common ratio만 저장하고 twisting factor를 runtime에 생성한다고 설명한다.

그다음 single_on_fly_wrap.v를 보면 된다.

wrapper는 계산기가 아니라 주로:

q ROM

twist seed ROM

tile 위치 tracking

다음 tile ROM prefetch

shadow register

start pulse 생성

을 담당하는 **controller + data supply layer**야.

여기서는 tb_single_on_fly16_general.v.

이 TB는

wrap/sof_in_general.hex

wrap/sof_exp_wrap_jlane_rot0_lr0.hex

를 읽고 **4x4 tiles = 256 frames**를 continuous하게 넣어서 golden과 비교한다.

끝에

[PASS] single_on_fly_wrap general (4x4 tiles)

가 나오면 된다.

9. 이제서야 phase1_256pe

여기 오면 파일이 갑자기 쉬워져.

왜냐하면 실제 내용이 거의:

U_NTT16_COL

↓
U_TRP
↓
U_SOF
↓
U_NTT16_ROW

이거니까.

여기서는 **tb_phase1_256pe_dc.v**부터 추천한다.

이 TB는 모든 lane에 그냥

1

을 계속 집어넣어.

그리고 각각:

phase1_vector/in.hex

phase1_vector/col_ntt.hex

phase1_vector/transpose.hex

phase1_vector/sof.hex

phase1_vector/row_ntt.hex

를 저장한다.

중요하게도 이 TB는 **PASS/FAIL golden checker**가 아니야.

대신 "각 stage를 데이터가 제대로 통과하고 있나?"를 관찰하기 굉장히 좋아.

Wave에는 딱:

in_valid

DUT.U_NTT16_COL.m_valid

DUT.U_TRP.m_valid

DUT.U_SOF.out_valid

out_valid

DUT.tr_cnt

DUT.tile_r

DUT.tile_c

를 놓으면 돼.

그러면 실제 NTTU pipeline이 눈에 보인다.

10. 그다음 tb_phase1_256pe_gen

이건 규모를 실제 구조까지 키운 버전이야.

$$16 \times 16 \text{ tiles} \times 16 \text{ rows/tile} = 4096 \text{ frames}$$

를 처리한다.

한 frame에 16 coefficient니까:

$$4096 \times 16 = 65536 = 2^{16}$$

즉 여기서 드디어 **residue polynomial 하나의 2^{16} coefficient 전체**와 연결된다.

애도 각 stage를 파일로 dump하고 처음 입력부터 각 stage 첫 valid가 언제 나오는지 latency도 측정한다.

그래서 phase1_dc로 구조 보고 → phase1_gen으로 full stream 돌리는 순서가 좋다.

11. Phase 2는 double_on_fly가 새로운 부분

phase2_256pe 자체는 Phase 1과 거의 똑같아.

NTT16

↓

transpose

↓

double_on_fly

↓

NTT16

차이는 single이 double이 된 거야.

double_on_fly_wrap을 보면 seed가:

t0_even

t1_odd

h2

h2pre

처럼 더 많고, tile_c를 기준으로 다음 set을 prefetch해.

이게 SHARP에서 Phase 2의 twisting sequence가 더 복잡해서 **double OF-Twist**를 둔다는 부분과 연결된다. SHARP는 Phase 1과 달리 Phase 2에서 common ratios 자체도 geometric sequence가 되도록 구성하여 specialized double OF-Twist를 사용한다고 설명한다.

현재 네가 보낸 파일에는 **Phase2 전용 TB가 없어.**

그러니까 Phase1까지 이해한 뒤에는 Phase1 TB 패턴을 복사해서:

NTT_A output

transpose output

DoF output

NTT_B output

네 지점을 검사하는 TB가 다음 단계야.

12. 마지막이 top_2dntt_256x256

Top은:

phase1

↓

global_xpose_pp_256x256

↓

phase2

다.

여기서 global_xpose가 local transpose보다 훨씬 커.

Local transpose는:

$$16 \times 16$$

tile 하나.

Global transpose는:

$$256 \times 256$$

전체야.

그리고 global transpose에는 **A/B 두 buffer set × 각 16-bank SRAM**을 뒤서,

처음:

A fill

그 다음:

A read → Phase2

B fill ← Phase1

그 다음:

B read → Phase2

A fill ← Phase1

식의 ping-pong을 한다.

처음에는 4096 frame을 다 모아야 하기 때문에 warm-up이 크고, 이후부터 overlap된다.

13. 그런데 top에는 하나 체크해야 할 부분이 있어

현재 네가 보낸 phase2_256pe.v의 interface에는

```
input tile_r;
```

```
input tile_c;
```

가 있는데,

top_2dntt_256x256.v의 U_P2 instantiate에서는 이 두 port를 연결하지 않고 있어.

특히 Phase2 내부에서 tile_c는 bank가 바뀔 때:

```
tilec_cur <= tile_c;
```

에 사용돼.

그래서 **bank 0** 하나만 reset부터 돌릴 때는 tilec_cur=0으로 시작하므로 지나갈 수 있지만, 여러 RNS bank를 top에서 연속 처리하면 이 부분은 반드시 확인해야 한다.

즉 top TB를 바로 돌리기 전에 이 interface를 사수에게 물어보는 게 좋아.

“phase2의 tile_c는 top에서 일부러 open한 건가요? bank change 때 tilec_cur initialization에 사용되는 것 같은데요.”

이건 꽤 좋은 코드리뷰 질문이야.

14. ModelSim compile은 이렇게 분리해서 해

BConv 폴더의 barrett_mult.v, shoup_mult.v랑 지금 NTT 폴더 버전을 동시에 같은 work에 넣지 않는 걸 추천해. 이름이 같은 module이 있어서 어느 버전을 물고 가는지 꼬일 수 있어.

NTT 폴더 안의 버전만 놓고 dependency 순서대로:

```
vlib work
```

vlog modules.v

vlog barrett_mult.v

vlog shoup_mult.v

vlog radix2_butterfly_dif.v

vlog q_const_rom_bank.v

vlog twiddle16_rom_multibank.v

vlog ntt16_buscore.v

vlog transpose16x16_bus.v

vlog single_twist_rom.v

vlog single_on_fly.v

vlog single_on_fly_wrap.v

vlog double_twist_rom.v

vlog double_on_fly.v

vlog double_on_fly_wrap.v

vlog phase1_256pe.v

vlog phase2_256pe.v

vlog top_2dntt_256x256.v

ntt_params.vh는 별도 compile 대상이 아니라 include되는 header야.

그리고 TB를 하나씩 추가해서 돌리면 된다.

그래서 내가 네 입장이면 당장 이렇게 시작한다

오늘은 **ntt_params.vh 10분** → **radix2_butterfly_dif + TB** → **ntt16_buscore + TB** → **transpose + TB**까지만 한다.

그게 완전히 이해된 다음 **single OF-Twist** → **Phase1**로 올라간다.

가장 중요한 건 지금 **top_2dntt**를 처음부터 이해하려고 하지 않는 것이야. 지금 코드의 핵심 난이도는 사실 크게 세 개밖에 없어.

radix-2 NTT pipeline
transpose/data ordering
on-the-fly twiddle generation

이 셋을 각각 단독 TB로 이해하고 나면 Phase1/Phase2/top은 이미 이해한 블록들을 연결한 배관처럼 보이기 시작할 거야.

그리고 BTS와 연결해서 보면, BTS 역시 NTTU를 fully pipelined radix-2 구조로 만들고 twiddle 저장량을 줄이기 위해 on-the-fly twiddling을 사용한다. 다만 BTS의 전체 NTT mapping은 3D-NTT + PE-PE NoC이고, 지금 RTL의 최종 dataflow는 그걸 그대로 옮긴 게 아니라 tile/hierarchical 구조로 재구성된 것이다.